VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

SOCIAL MEDIA DATA ANALYSER

A9513 – PYTHON PROGRAMING LAB

Submitted by

PULIPATI BHARATHA SIMHA
25881A0576

GUNJI. KASTURI
25881A0588

Course Facilitator

Ms S Madhavi
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218, Telangana, India.

Vibrant Vardhaman
25 Years of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Social Media Analyser"** is carried out by **PULIPATI. BHARATHA SIMHA** and **GUNJI KASTURI** with Roll Numbers **25881A0576** and **25881A0588** towards **A9513 – PYTHON PROGRAMING Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty
Ms Pallamoni S Madhavi
Assistant Professor, CSE

Signature of the HOD
Dr. Y.Vijayalata
Professor,HOD, CSE

Contents

1 Introduction 3
2 Literature Review 3
3 Objectives 3
4 Methodology 3
4.1 Architectural Diagram 3
4.2 Hardware and Software Requirements 3
4.3 Working Principle 4
4.4 Software Implementation 4
5 Results and Discussion 4
6 Mapping POs , P S O s and SDGs 5
6.1 Program Outcomes (POs) Mapping 5
6.2 Sustainable Development Goals (SDGs) Mapping 5
7 Conclusion 5
Bibliography 6

Abstract

The Social Media Analyser is a desktop application developed in Python using the Tkinter library and its themed widget module, ttk. It addresses the difficulty of tracking the likes, comments and shares of many posts published by many users, where manual tracking easily leads to repeated IDs, missing details and calculation mistakes. The application keeps users and posts in an organised form in which every user has a unique ID and every post belongs to a particular user. Its window has four tabs, namely Users, Posts, Analyse and Individual Report, through which new users and posts can be added and displayed in tables, a single post can be analysed, and a report of all the posts of a user can be viewed.

Internally, the program is written using Object-Oriented Programming. The classes SocialMediaPost and User store the data, SocialMediaAnalyser manages the collections of users and posts using dictionaries, and SocialMediaAnalyserApp builds the graphical interface, while the custom exception SocialMediaError reports empty fields, negative values, duplicate IDs and unknown users. The total engagement of a post is calculated as the sum of its likes, comments and shares. A post is classified as Poor, Average, Good or Excellent based on its likes, and its engagement status is decided as High, Medium or Low based on its total engagement. The individual report shows the details of every post of a user together with the total likes, average likes, total comments and total shares. Predefined users and posts are stored inside the program, so the application can be run directly, and only the Python standard library is used.

Keywords: Social Media Analyser, Python, Tkinter, Engagement Analysis, Object-Oriented Programming, Exception Handling, Performance Classification, Graphical User Interface

1 Introduction

Social media has become a regular part of everyday life. Platforms such as Instagram, Facebook and LinkedIn allow people to share short messages, photos and ideas with others. Every post that is published receives reactions from the audience in the form of likes, comments and shares. These reactions are simple numbers, but when they are collected and studied together they give a useful idea of how well a post has been received.

In this project, social media analysis means collecting the basic details of users and their posts and then using simple calculations to understand how each post and each user is performing. The project does not deal with large amounts of data or advanced analytics. It works with a small set of values (likes, comments and shares) and applies clear, rule-based logic to them.

When many users publish many posts, the information becomes difficult to track manually. Keeping the data in an organised form, where every post belongs to a particular user and every user has a unique ID, makes it easy to find a post, count the posts of a user and prepare summaries. Organised data also reduces mistakes such as repeated IDs or missing details.

Engagement metrics help to measure the response of the audience. Likes show quick appreciation, comments show that viewers cared enough to respond, and shares show that the content was considered worth passing on to others. Adding these three values gives the total engagement of a post, which is a single number that can be used to compare posts with each other.

The **Social Media Analyser** is a desktop application developed in Python using the Tkinter library and its themed widget module, ttk. The application window has four tabs: **Users**, **Posts**, **Analyse** and **Individual Report**. In the Users and Posts tabs, new users and posts can be added and are displayed in tables. In the Analyse tab, a post can be selected to view its likes, comments, shares, total engagement, a likes-based performance category and an engagement status. In the Individual Report tab, a user can be selected to view the details of all of his or her posts together with summary values such as total likes, average likes, total comments and total shares.

The program is written using Object-Oriented Programming. Separate classes represent a post, a user, the analysing system and the GUI window, and a custom exception class is used to report invalid input. Some users and posts are already stored inside the program, so the application can be run directly without any separate data file.

2 Literature Review

This project brings together several basic topics that are studied in a first-year Python programming course. The following sections describe the concepts that are relevant to the Social Media Analyser and how each of them is used in the application. Python is a high-level, general-purpose programming language that is known for its simple and readable syntax. It provides built-in data types such as numbers, strings, lists and dictionaries, and a standard library that supports many common tasks [8], [9], [10]. Because of this, a complete desktop application can be written in Python without installing any additional package. The Social Media Analyser uses only the Python standard library, namely the Tkinter GUI modules Object-Oriented Programming (OOP) organises a program into classes and objects. A class groups related data (attributes) and the functions that work on that data (methods), and a constructor method, __init__, prepares each new object [4]. Python also supports properties, which allow a value to be calculated when it is accessed, and inheritance, which allows a new class to be built on an existing one [4], [7]. In this project, SocialMediaPost and User store the data, SocialMediaAnalyser manages the collections of users and posts, the exception class SocialMediaError inherits from the built-in Exception class, and the application window class inherits from tk.Tk.

Tkinter is the standard Python interface to the Tk GUI toolkit [1]. It provides widgets such as labels, entry boxes, buttons and text areas. The ttk module adds themed widgets such as Notebook (tabs), Combobox (drop-down lists), Treeview (tables) and LabelFrame (grouped sections) [2]. A Tkinter program is event-driven: the window waits in a loop for the user to act, and a function is called when a button is pressed. The messagebox module is used to display information, warning and error dialogs [3]. Detailed explanations of building applications with Tkinter are also available in the book by Grayson [11]. The Social Media Analyser uses all of these components to build its four-tab interface. Python lists store items in order, while dictionaries store values against keys and allow quick lookup [6]. Both are used in this project. The analyser keeps its users and posts in two dictionaries, with the user ID and post ID as keys. Each user keeps his or her posts in a list. The predefined data is written as a dictionary that contains lists of dictionaries. String methods such as strip(), upper() and split() are used to clean the entered text and to read the ID from the selected drop-down item, and the built-in sum() function is used to calculate totals.

Exception handling allows a program to deal with errors without stopping unexpectedly. A program can raise an exception when it finds a problem and handle it using try and except blocks [5]. Programmers can also define their own exception classes so that errors of their own application can be separated from other errors [5]. In this project, SocialMediaError is raised for empty fields, negative values, duplicate IDs and unknown users, and the GUI catches it and displays the message to the user. A ValueError is also handled when likes, comments or shares are not entered as numbers

.

3 Objectives

- **To design and develop a Java-based scheduling simulator supporting multiple CPU scheduling algorithms.**

The primary objective of this project is to build a functional and efficient CPU scheduling simulator using Java. The system will incorporate commonly used scheduling algorithms such as First-Come-First-Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round-Robin (RR). The simulator aims to mimic the behavior of real operating system schedulers by managing process arrival order, CPU burst times, and priority assignments.

- **To implement GUI-based process input, execution visualization, and Gantt chart generation.**

A user-friendly graphical interface is essential to help students and users easily interact with the simulator. This objective focuses on creating a GUI that allows users to input process details such as process ID, arrival time, burst time, and priority. The system will dynamically simulate process execution and visually display scheduling sequences through Gantt charts. This visual representation helps users understand how processes are scheduled, how CPU time is distributed, and how each algorithm behaves in real time.

- **To calculate and compare performance metrics like turnaround time and waiting time for each algorithm.**

This objective involves computing essential scheduling metrics including turnaround time, waiting time, response time, and throughput (if applicable) for each simulated process. The simulator will analyze and compare the output of different algorithms to demonstrate performance variations based on process characteristics and scheduling logic. By presenting calculated metrics in tabular or graphical form, users can better grasp the trade-offs between fairness, efficiency, and responsiveness, reinforcing theoretical knowledge through practical comparison.

4 Methodology

4.1 Architectural Diagram

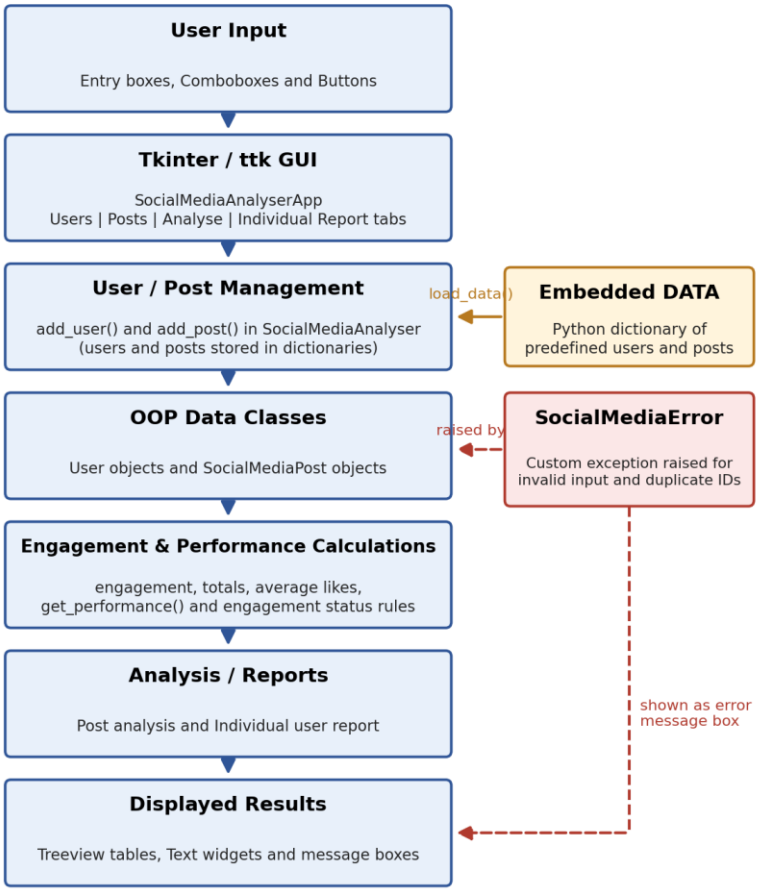


Figure 1: Architectural Diagram

The Social Media Analyser is a standalone desktop application. All of its parts run inside a single Python program, and no database, server, network connection or cloud service is used. The flow of data through the application

User Input: The user types values into entry boxes, selects a user or a post from drop-down lists and presses buttons such as Add User, Add Post, Analyse and View Report.

Tkinter / ttk GUI**:** The class SocialMediaAnalyserApp builds the window with four tabs and calls the required method when a button is pressed.

User / Post Management: The class SocialMediaAnalyser receives the data from the GUI, validates it, and stores users and posts in dictionaries. The same methods are used to load the predefined data (the embedded DATA) when the program starts.

OOP Data Classes: Every user is stored as a User object and every post as a SocialMediaPost object. A user object keeps the list of its own posts.

Engagement and Performance Calculations: The engagement of a post, the totals of a user, the average likes and the performance category (get_performance()) are calculated here, and the engagement status is decided from the total engagement.

Analysis / Reports: The post analysis and the individual user report are prepared as formatted text.

Displayed Results: The results are shown in the Treeview tables, in the Text areas of the Analyse and Individual Report tabs, and in message boxes. Errors raised as SocialMediaError are also shown to the user through message boxes.

4.2 Hardware and Software Requirements

Hardware:

- Processor
- 2-GB RAM
- Display 1024*768

Software:

- Programming language Python 3
- GUI library Thinker and ttk
- Operating Systems

4.3 Working Principle

The program starts from the if __name__ == "__main__": block, which creates an object of SocialMediaAnalyserApp and calls mainloop(). While creating the object, the constructor creates a SocialMediaAnalyser object, loads the embedded data, sets the window title, size and minimum size, builds the GUI and finally refreshes all tables and drop-down lists.

The program contains a dictionary named DATA with six predefined users (U001 to U006) and four predefined posts (P001, P002, P003 and P005). The method load_data() adds every user and then every post to the analyser using the same add_user() and add_post() methods that are used later by the GUI. Because of this, the predefined data is checked by the same validation rules as any data entered by the user. A ttk.Notebook is created with four tabs: Users, Posts, Analyse and Individual Report. Each tab is built by its own method.

The tab contains a form with User ID, Name and Username fields and an Add User button. Below the form, a table lists the User ID, Name, Username, number of posts, total likes and total engagement of every user. When Add User is pressed, the new user is created and stored, a success message is shown, the entry boxes are cleared and the table is refreshed.

The form contains a drop-down list of users, and fields for Post ID, Content, Platform, Likes, Comments and Shares. The drop-down items have the form “U001 - Kasturi (@kasturi)”, and the user ID is taken from the part before the “ - ” separator. After the likes, comments and shares are converted to numbers, the post is created and linked to the selected user. The table below the form lists every post along with its performance category, which is calculated from the likes at the time of display.

The user selects a post from a drop-down list whose items have the form “P001 - Learning Python is fun!” and presses Analyse. The program finds the post using its ID and displays its details, its total engagement, the likes-based performance category and the engagement status in a read-only text area.

The user selects a user from a drop-down list and presses View Report. The report shows the name, username and user ID, then the details and performance category of each post of that user, and finally a user summary. If the user has no posts, the message “No posts found for this user.” is displayed.

The following calculations are performed by the program:

- Total engagement of a post = likes + comments + shares.
- Total likes, total comments, total shares and total engagement of a user = the sum of the corresponding values of all posts of that user.
- Average likes of a user = total likes ÷ number of posts of that user.
- Likes-based performance category of a post (from its likes), and of a user (from his or her average likes), using the limits in Table 3.
- Engagement status of a post (from its total engagement), using the limits in Table 4.

Example problem:

User ID	Posts	Likes	Comments
U1	25	125000	1,25
U2	18	75000	750
U3	12	35000	35,00
U4	8	75	75
U5	10	1,50,000	457

Performance Classifiaction:
-> Likes > 1,00,000 ->Excellen**t**
-> Likes > 50,000 ->Good
-> Likes > 5000 ->Average
-> Likes > 100 ->Poor

User Performance Report:

- U1 → 1,25,000 likes → Excellent
- U2 → 75,000 likes → Good
- U3 → 35,000 likes → Average
- U4 → 75 likes → Poor
- U5 → 1,50,000 likes → Excellent

The system successfully analyses the selected user's social media activity and generates a performance report based on the number of likes received.

4.4 Software Implementation

```
import json
import tkinter as tk
from tkinter import ttk, messagebox


# ============================================================
# SOCIAL MEDIA ANALYSER - SINGLE VS CODE FILE
# ============================================================
# This version combines:
# 1. OOP based Social Media Analyser
# 2. Users and Posts management
# 3. Post analysis
# 4. Overall report
# 5. Individual user report
# 6. Likes-based performance:
#       Below 100       -> Poor
#       100 to 50,000   -> Average
#       50,001 to 100k  -> Good
#       Above 100,000   -> Excellent
#
# No separate JSON file is required. The data is included below.
# ============================================================


class SocialMediaError(Exception):
    pass


class SocialMediaPost:
    def __init__(self, post_id, content, platform, likes, comments, shares):
        if not post_id.strip():
            raise SocialMediaError("Post ID cannot be empty.")
        if not content.strip():
            raise SocialMediaError("Post content cannot be empty.")
        if not platform.strip():
            raise SocialMediaError("Platform cannot be empty.")
        if likes < 0 or comments < 0 or shares < 0:
            raise SocialMediaError("Likes, comments and shares cannot be negative.")

```

```
        self.post_id = post_id.strip().upper()
        self.content = content.strip()
        self.platform = platform.strip()
        self.likes = likes
        self.comments = comments
        self.shares = shares

    @property
    def engagement(self):
        return self.likes + self.comments + self.shares


class User:
    def __init__(self, user_id, name, username):
        if not user_id.strip():
            raise SocialMediaError("User ID cannot be empty.")
        if not name.strip():
            raise SocialMediaError("Name cannot be empty.")
        if not username.strip():
            raise SocialMediaError("Username cannot be empty.")

        self.user_id = user_id.strip().upper()
        self.name = name.strip()
        self.username = username.strip()
        self.posts = []

    def add_post(self, post):
        self.posts.append(post)

    @property
    def total_likes(self):
        return sum(post.likes for post in self.posts)

    @property
    def total_comments(self):
        return sum(post.comments for post in self.posts)

    @property
    def total_shares(self):
        return sum(post.shares for post in self.posts)

    @property
```

```
    def total_engagement(self):
        return self.total_likes + self.total_comments + self.total_shares


class SocialMediaAnalyser:
    def __init__(self):
        self.users = {}
        self.posts = {}

    def add_user(self, user_id, name, username):
        user = User(user_id, name, username)
        if user.user_id in self.users:
            raise SocialMediaError(f"User ID '{user.user_id}' already exists.")
        self.users[user.user_id] = user
        return user

    def add_post(self, user_id, post_id, content, platform, likes, comments, shares):
        uid = user_id.strip().upper()
        if uid not in self.users:
            raise SocialMediaError(f"No user found with ID '{uid}'.")

        post = SocialMediaPost(
            post_id, content, platform, likes, comments, shares
        )

        if post.post_id in self.posts:
            raise SocialMediaError(f"Post ID '{post.post_id}' already exists.")

        self.posts[post.post_id] = post
        self.users[uid].add_post(post)
        return post

    def most_popular_post(self):
        if not self.posts:
            raise SocialMediaError("No posts available for analysis.")
        return max(self.posts.values(), key=lambda post: post.engagement)


# ============================================================
# LIKES PERFORMANCE RULE
# ============================================================

```

```
def get_performance(likes):
    """Return the report category based on likes."""
    if likes < 100:
        return "Poor"
    elif likes <= 50000:
        return "Average"
    elif likes <= 100000:
        return "Good"
    else:
        return "Excellent"


# ============================================================
# ORIGINAL USER DATA FROM THE PROVIDED JSON
# ============================================================

DATA = {
  "users": [
    {
      "user_id": "U001",
      "name": "Kasturi",
      "username": "@kasturi"
    },
    {
      "user_id": "U002",
      "name": "Rahul",
      "username": "@rahul"
    },
    {
      "user_id": "U003",
      "name": "Ananya",
      "username": "@ananya"
    },
    {
      "user_id": "U004",
      "name": "madhu",
      "username": "@madhu"
    },
    {
      "user_id": "U005",
      "name": "Ramya",
      "username": "@ramya"
```

```
    },
    {
      "user_id": "U006",
      "name": "NANDINI",
      "username": "@NAN"
    }
  ],
  "posts": [
    {
      "post_id": "P001",
      "content": "Learning Python is fun!",
      "platform": "Instagram",
      "likes": 250,
      "comments": 35,
      "shares": 20,
      "user_id": "U001"
    },
    {
      "post_id": "P002",
      "content": "My first Tkinter project",
      "platform": "Facebook",
      "likes": 180,
      "comments": 25,
      "shares": 15,
      "user_id": "U002"
    },
    {
      "post_id": "P003",
      "content": "Exploring AI and Data Science",
      "platform": "Instagram",
      "likes": 420,
      "comments": 70,
      "shares": 45,
      "user_id": "U003"
    },
    {
      "post_id": "P005",
      "content": "EXTITED TO SHARE MY EXPO",
      "platform": "LinkedIn",
      "likes": 127,
      "comments": 54,
```

```
        "shares": 7,
        "user_id": "U006"
    }
]
}

# ============================================================
# GUI APPLICATION
# ============================================================

class SocialMediaAnalyserApp(tk.Tk):
    def __init__(self):
        super().__init__()

        self.system = SocialMediaAnalyser()
        self.load_data()

        self.title("Social Media Analyser")
        self.geometry("1000x700")
        self.minsize(850, 600)

        self.build_gui()
        self.refresh_all()

    # ---------------------------------------------------
    # Load embedded data
    # ---------------------------------------------------
    def load_data(self):
        for user in DATA["users"]:
            self.system.add_user(
                user["user_id"],
                user["name"],
                user["username"]
            )

        for post in DATA["posts"]:
            self.system.add_post(
                post["user_id"],
                post["post_id"],
                post["content"],
                post["platform"],
```

```
            post["likes"],
            post["comments"],
            post["shares"]
        )

# ---------------------------------------------------------
# GUI
# ---------------------------------------------------------
def build_gui(self):
    style = ttk.Style()
    style.configure("Title.TLabel", font=("Arial", 18, "bold"))
    style.configure("TButton", font=("Arial", 10, "bold"), padding=6)

    notebook = ttk.Notebook(self)
    notebook.pack(fill="both", expand=True, padx=8, pady=8)

    self.users_tab = ttk.Frame(notebook)
    self.posts_tab = ttk.Frame(notebook)
    self.analyse_tab = ttk.Frame(notebook)
    self.report_tab = ttk.Frame(notebook)

    notebook.add(self.users_tab, text="Users")
    notebook.add(self.posts_tab, text="Posts")
    notebook.add(self.analyse_tab, text="Analyse")
    notebook.add(self.report_tab, text="Individual Report")

    self.build_users_tab()
    self.build_posts_tab()
    self.build_analyse_tab()
    self.build_report_tab()

# ---------------------------------------------------------
# USERS TAB
# ---------------------------------------------------------
def build_users_tab(self):
    frame = self.users_tab

    form = ttk.LabelFrame(frame, text="Add Social Media User")
    form.pack(fill="x", padx=10, pady=10)

    ttk.Label(form, text="User ID").grid(row=0, column=0, padx=5, pady=5)
```

```
286     self.uid_entry = ttk.Entry(form)
287     self.uid_entry.grid(row=0, column=1, padx=5, pady=5)
288
289     ttk.Label(form, text="Name").grid(row=0, column=2, padx=5, pady=5)
290     self.uname_entry = ttk.Entry(form)
291     self.uname_entry.grid(row=0, column=3, padx=5, pady=5)
292
293     ttk.Label(form, text="Username").grid(row=0, column=4, padx=5, pady=5)
294     self.username_entry = ttk.Entry(form)
295     self.username_entry.grid(row=0, column=5, padx=5, pady=5)
296
297     ttk.Button(form, text="Add User", command=self.add_user).grid(
298         row=0, column=6, padx=10, pady=5
299     )
300
301     columns = ("id", "name", "username", "posts", "likes", "engagement")
302     self.users_table = ttk.Treeview(frame, columns=columns, show="headings")
303
304     headings = {
305         "id": "User ID",
306         "name": "Name",
307         "username": "Username",
308         "posts": "Posts",
309         "likes": "Total Likes",
310         "engagement": "Total Engagement"
311     }
312
313     for column in columns:
314         self.users_table.heading(column, text=headings[column])
315         self.users_table.column(column, width=145, anchor="center")
316
317     self.users_table.pack(fill="both", expand=True, padx=10, pady=10)
318
319 def add_user(self):
320     try:
321         user = self.system.add_user(
322             self.uid_entry.get(),
323             self.uname_entry.get(),
324             self.username_entry.get()
325         )
326         messagebox.showinfo("Success", f"User added: {user.name}")
```

```
            self.uid_entry.delete(0, tk.END)
            self.uname_entry.delete(0, tk.END)
            self.username_entry.delete(0, tk.END)
            self.refresh_all()
        except SocialMediaError as error:
            messagebox.showerror("Error", str(error))

    def refresh_users_table(self):
        self.users_table.delete(*self.users_table.get_children())

        for user in self.system.users.values():
            self.users_table.insert(
                "",
                "end",
                values=(
                    user.user_id,
                    user.name,
                    user.username,
                    len(user.posts),
                    user.total_likes,
                    user.total_engagement
                )
            )

    # ----------------------------------------------------------
    # POSTS TAB
    # ----------------------------------------------------------
    def build_posts_tab(self):
        frame = self.posts_tab

        form = ttk.LabelFrame(frame, text="Add Social Media Post")
        form.pack(fill="x", padx=10, pady=10)

        ttk.Label(form, text="User").grid(row=0, column=0, padx=5, pady=5)
        self.post_user_cb = ttk.Combobox(form, state="readonly", width=25)
        self.post_user_cb.grid(row=0, column=1, padx=5, pady=5)

        ttk.Label(form, text="Post ID").grid(row=0, column=2, padx=5, pady=5)
        self.post_id_entry = ttk.Entry(form)
        self.post_id_entry.grid(row=0, column=3, padx=5, pady=5)

```

```
368     ttk.Label(form, text="Content").grid(row=1, column=0, padx=5, pady=5)
369     self.content_entry = ttk.Entry(form, width=40)
370     self.content_entry.grid(row=1, column=1, columnspan=3, padx=5, pady=5)
371
372     ttk.Label(form, text="Platform").grid(row=2, column=0, padx=5, pady=5)
373     self.platform_entry = ttk.Entry(form)
374     self.platform_entry.grid(row=2, column=1, padx=5, pady=5)
375
376     ttk.Label(form, text="Likes").grid(row=2, column=2, padx=5, pady=5)
377     self.likes_entry = ttk.Entry(form)
378     self.likes_entry.grid(row=2, column=3, padx=5, pady=5)
379
380     ttk.Label(form, text="Comments").grid(row=3, column=0, padx=5, pady=5)
381     self.comments_entry = ttk.Entry(form)
382     self.comments_entry.grid(row=3, column=1, padx=5, pady=5)
383
384     ttk.Label(form, text="Shares").grid(row=3, column=2, padx=5, pady=5)
385     self.shares_entry = ttk.Entry(form)
386     self.shares_entry.grid(row=3, column=3, padx=5, pady=5)
387
388     ttk.Button(form, text="Add Post", command=self.add_post).grid(
389         row=4, column=1, columnspan=2, padx=10, pady=10
390     )
391
392     columns = (
393         "id", "content", "platform", "likes",
394         "comments", "shares", "performance"
395     )
396
397     self.posts_table = ttk.Treeview(frame, columns=columns, show="headings")
398
399     headings = {
400         "id": "Post ID",
401         "content": "Content",
402         "platform": "Platform",
403         "likes": "Likes",
404         "comments": "Comments",
405         "shares": "Shares",
406         "performance": "Performance"
407     }
408
```

```
409     for column in columns:
410         self.posts_table.heading(column, text=headings[column])
411         self.posts_table.column(column, width=120, anchor="center")
412 
413     self.posts_table.pack(fill="both", expand=True, padx=10, pady=10)
414 
415 def add_post(self):
416     try:
417         user_value = self.post_user_cb.get()
418         if not user_value:
419             raise SocialMediaError("Please select a user.")
420 
421         user_id = user_value.split(" - ")[0]
422 
423         likes = int(self.likes_entry.get())
424         comments = int(self.comments_entry.get())
425         shares = int(self.shares_entry.get())
426 
427         post = self.system.add_post(
428             user_id,
429             self.post_id_entry.get(),
430             self.content_entry.get(),
431             self.platform_entry.get(),
432             likes,
433             comments,
434             shares
435         )
436 
437         messagebox.showinfo("Success", f"Post added: {post.post_id}")
438 
439         for entry in (
440             self.post_id_entry,
441             self.content_entry,
442             self.platform_entry,
443             self.likes_entry,
444             self.comments_entry,
445             self.shares_entry
446         ):
447             entry.delete(0, tk.END)
448 
449         self.refresh_all()
```

```
450
451         except ValueError:
452             messagebox.showerror(
453                 "Error", "Likes, comments and shares must be numbers."
454             )
455         except SocialMediaError as error:
456             messagebox.showerror("Error", str(error))
457
458     def refresh_posts_table(self):
459         self.posts_table.delete(*self.posts_table.get_children())
460
461         for post in self.system.posts.values():
462             self.posts_table.insert(
463                 "",
464                 "end",
465                 values=(
466                     post.post_id,
467                     post.content,
468                     post.platform,
469                     post.likes,
470                     post.comments,
471                     post.shares,
472                     get_performance(post.likes)
473                 )
474             )
475
476     # ----------------------------------------------------------
477     # ANALYSE TAB
478     # ----------------------------------------------------------
479     def build_analyse_tab(self):
480         frame = self.analyse_tab
481
482         form = ttk.LabelFrame(frame, text="Analyse Social Media Post")
483         form.pack(fill="x", padx=10, pady=10)
484
485         ttk.Label(form, text="Select Post").grid(
486             row=0, column=0, padx=5, pady=10
487         )
488
489         self.analyse_post_cb = ttk.Combobox(
490             form, state="readonly", width=50
```

```
491         )
492         self.analyse_post_cb.grid(row=0, column=1, padx=5, pady=10)
493
494         ttk.Button(
495             form, text="Analyse", command=self.analyse_post
496         ).grid(row=0, column=2, padx=10)
497
498         self.analysis_text = tk.Text(
499             frame, height=20, font=("Courier New", 11)
500         )
501         self.analysis_text.pack(
502             fill="both", expand=True, padx=10, pady=10
503         )
504         self.analysis_text.config(state="disabled")
505
506     def analyse_post(self):
507         value = self.analyse_post_cb.get()
508         if not value:
509             messagebox.showwarning("Select Post", "Please select a post.")
510             return
511
512         post_id = value.split(" - ")[0]
513         post = self.system.posts[post_id]
514
515         status = (
516             "HIGH ENGAGEMENT" if post.engagement >= 300
517             else "MEDIUM ENGAGEMENT" if post.engagement >= 150
518             else "LOW ENGAGEMENT"
519         )
520
521         result = [
522             "=" * 60,
523             "SOCIAL MEDIA POST ANALYSIS",
524             "=" * 60,
525             f"Post ID      : {post.post_id}",
526             f"Content      : {post.content}",
527             f"Platform     : {post.platform}",
528             "-" * 60,
529             f"Likes        : {post.likes:,}",
530             f"Comments     : {post.comments:,}",
531             f"Shares       : {post.shares:,}",
```

```
        f"Total        : {post.engagement:,}",
        f"Likes Report  : {get_performance(post.likes)}",
        "-" * 60,
        f"Engagement Status : {status}",
        "=" * 60
    ]

    self.analysis_text.config(state="normal")
    self.analysis_text.delete("1.0", tk.END)
    self.analysis_text.insert(tk.END, "\n".join(result))
    self.analysis_text.config(state="disabled")

# ---------------------------------------------------------
# INDIVIDUAL USER REPORT TAB
# ---------------------------------------------------------
def build_report_tab(self):
    frame = self.report_tab

    title = ttk.Label(
        frame,
        text="INDIVIDUAL USER REPORT",
        font=("Arial", 18, "bold")
    )
    title.pack(pady=(15, 5))

    info = ttk.Label(
        frame,
        text="Select an individual user to check their likes-based report."
    )
    info.pack(pady=(0, 10))

    selection = ttk.Frame(frame)
    selection.pack(fill="x", padx=15, pady=5)

    ttk.Label(
        selection,
        text="Select User:",
        font=("Arial", 11, "bold")
    ).pack(side="left", padx=(0, 10))

    self.report_user_cb = ttk.Combobox(
```

```
            selection, state="readonly", width=45
        )
        self.report_user_cb.pack(side="left", fill="x", expand=True)

        ttk.Button(
            selection,
            text="VIEW REPORT",
            command=self.show_user_report
        ).pack(side="left", padx=(10, 0))

        legend = ttk.Label(
            frame,
            text=(
                "Below 100 = Poor  |  100–50,000 = Average  |  "
                "50,001–100,000 = Good  |  Above 100,000 = Excellent"
            ),
            font=("Arial", 9)
        )
        legend.pack(pady=10)

        self.report_text = tk.Text(
            frame,
            wrap="word",
            font=("Consolas", 10),
            padx=12,
            pady=12
        )
        self.report_text.pack(
            fill="both", expand=True, padx=15, pady=10
        )

        self.report_text.tag_configure(
            "title", font=("Consolas", 14, "bold")
        )
        self.report_text.tag_configure(
            "subtitle", font=("Consolas", 11, "bold")
        )
        self.report_text.tag_configure(
            "poor", font=("Consolas", 10, "bold")
        )
        self.report_text.tag_configure(
```

```
        "average", font=("Consolas", 10, "bold")
    )
    self.report_text.tag_configure(
        "good", font=("Consolas", 10, "bold")
    )
    self.report_text.tag_configure(
        "excellent", font=("Consolas", 10, "bold")
    )

    self.report_text.config(state="disabled")

def show_user_report(self):
    selected = self.report_user_cb.get()

    if not selected:
        messagebox.showwarning(
            "Select User", "Please select a user first."
        )
        return

    user_id = selected.split(" - ")[0]
    user = self.system.users.get(user_id)

    if user is None:
        messagebox.showerror("Error", "User not found.")
        return

    posts = user.posts

    self.report_text.config(state="normal")
    self.report_text.delete("1.0", tk.END)

    self.report_text.insert(
        tk.END, "INDIVIDUAL USER REPORT\n", "title"
    )
    self.report_text.insert(tk.END, "=" * 60 + "\n")
    self.report_text.insert(tk.END, f"Name     : {user.name}\n")
    self.report_text.insert(tk.END, f"Username : {user.username}\n")
    self.report_text.insert(tk.END, f"User ID  : {user.user_id}\n")
    self.report_text.insert(tk.END, "-" * 60 + "\n")

```

```
655     if not posts:
656         self.report_text.insert(
657             tk.END, "No posts found for this user.\n"
658         )
659         self.report_text.config(state="disabled")
660         return
661 
662     total_likes = user.total_likes
663     total_comments = user.total_comments
664     total_shares = user.total_shares
665 
666     for index, post in enumerate(posts, start=1):
667         category = get_performance(post.likes)
668 
669         self.report_text.insert(
670             tk.END, f"POST {index}\n", "subtitle"
671         )
672         self.report_text.insert(
673             tk.END, f"Post ID     : {post.post_id}\n"
674         )
675         self.report_text.insert(
676             tk.END, f"Platform    : {post.platform}\n"
677         )
678         self.report_text.insert(
679             tk.END, f"Content     : {post.content}\n"
680         )
681         self.report_text.insert(
682             tk.END, f"Likes       : {post.likes:,}\n"
683         )
684         self.report_text.insert(
685             tk.END, f"Comments    : {post.comments:,}\n"
686         )
687         self.report_text.insert(
688             tk.END, f"Shares      : {post.shares:,}\n"
689         )
690         self.report_text.insert(tk.END, "Performance : ")
691         self.report_text.insert(
692             tk.END, category + "\n", category.lower()
693         )
694         self.report_text.insert(tk.END, "-" * 60 + "\n")
695 
```

```
696     average_likes = total_likes / len(posts)
697     overall_category = get_performance(average_likes)
698 
699     self.report_text.insert(
700         tk.END, "USER SUMMARY\n", "subtitle"
701     )
702     self.report_text.insert(tk.END, "=" * 60 + "\n")
703     self.report_text.insert(
704         tk.END, f"Total Posts     : {len(posts)}\n"
705     )
706     self.report_text.insert(
707         tk.END, f"Total Likes     : {total_likes:,}\n"
708     )
709     self.report_text.insert(
710         tk.END, f"Average Likes   : {average_likes:,.0f}\n"
711     )
712     self.report_text.insert(
713         tk.END, f"Total Comments  : {total_comments:,}\n"
714     )
715     self.report_text.insert(
716         tk.END, f"Total Shares    : {total_shares:,}\n"
717     )
718     self.report_text.insert(tk.END, "Overall Report  : ")
719     self.report_text.insert(
720         tk.END, overall_category + "\n", overall_category.lower()
721     )
722 
723     self.report_text.config(state="disabled")
724 
725 # ------------------------------------------------------------
726 # REFRESH ALL
727 # ------------------------------------------------------------
728 def refresh_all(self):
729     self.refresh_users_table()
730     self.refresh_posts_table()
731 
732     user_values = [
733         f"{user.user_id} - {user.name} ({user.username})"
734         for user in self.system.users.values()
735     ]
736     self.post_user_cb["values"] = user_values
```

```
        self.report_user_cb["values"] = user_values

        post_values = [
            f"{post.post_id} - {post.content}"
            for post in self.system.posts.values()
        ]
        self.analyse_post_cb["values"] = post_values

        if user_values:
            if not self.post_user_cb.get():
                self.post_user_cb.current(0)
            if not self.report_user_cb.get():
                self.report_user_cb.current(0)

        if post_values and not self.analyse_post_cb.get():
            self.analyse_post_cb.current(0)


# ============================================================
# RUN PROGRAM
# ============================================================

if __name__ == "__main__":
    app = SocialMediaAnalyserApp()
    app.mainloop()
```

Results and Discussion

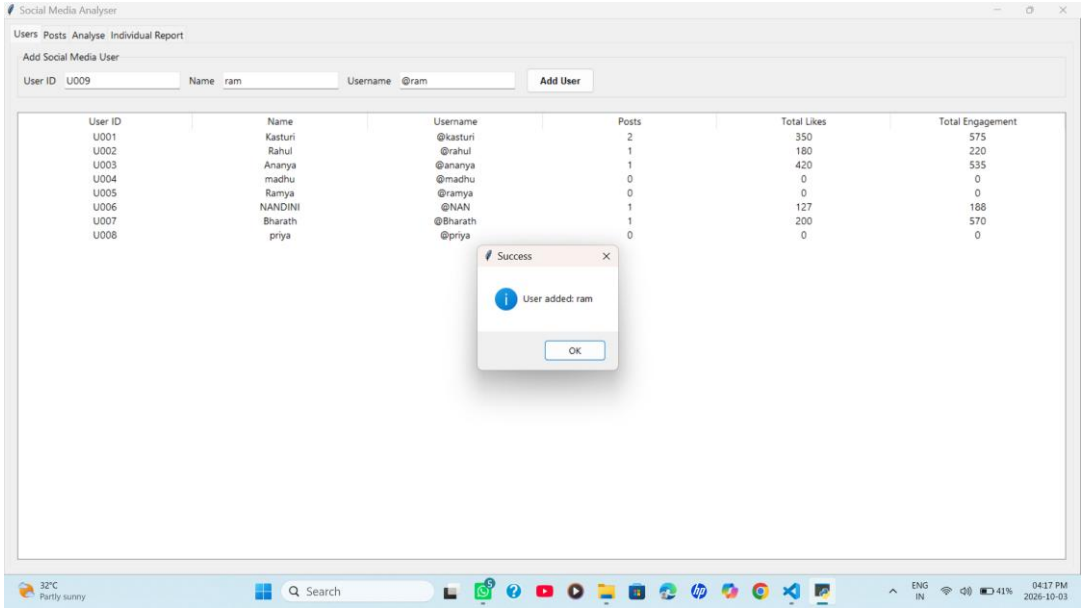


Figure 1 User Management Interface

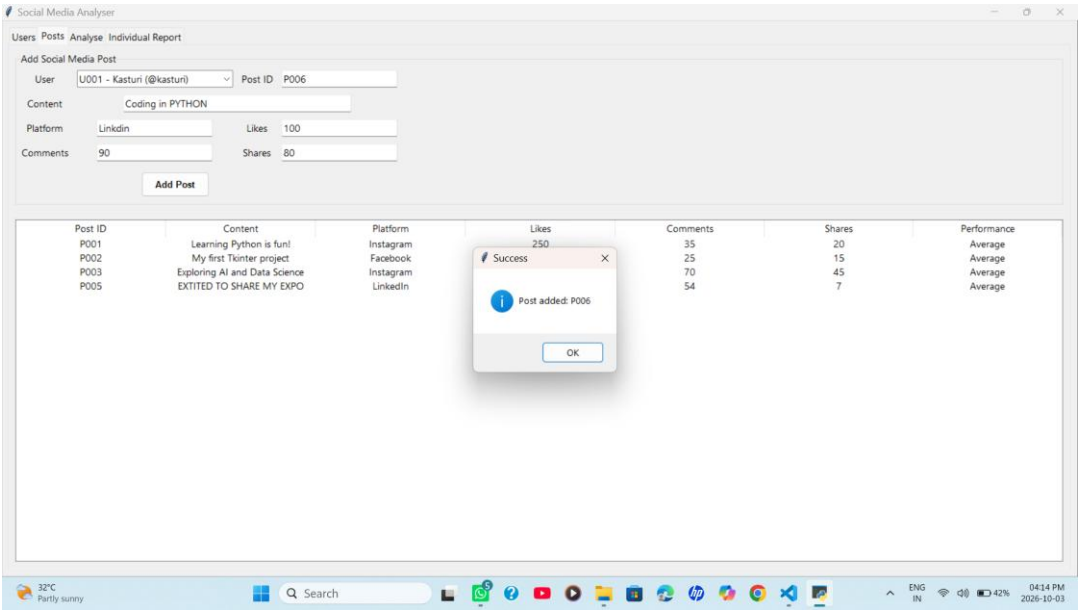


Figure 2 Social Media Post Management Interface

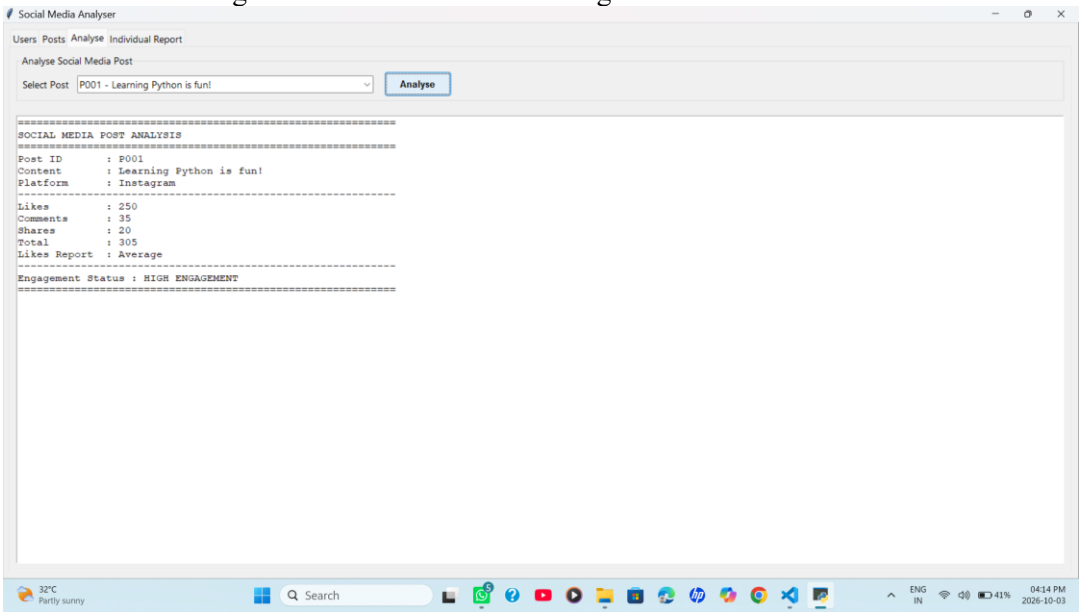


Figure 3 : Social Media Post Analysis

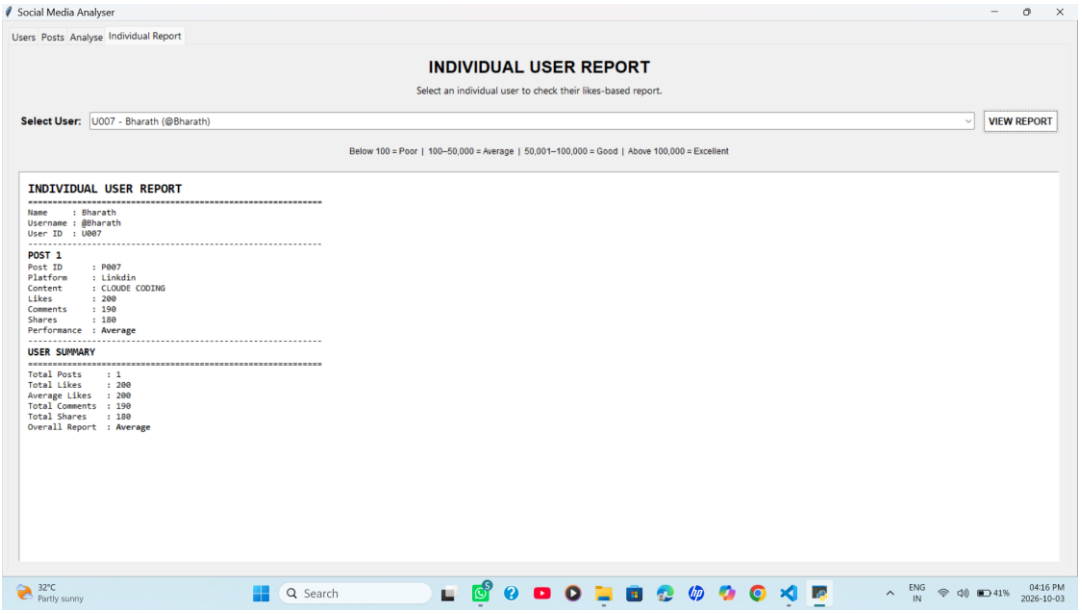


Figure 4 Social Media Data Individual report analysis

Summary of Results

The Social Media Data Analyser successfully processes social media data and generates individual user performance reports based on the number of likes. The system classifies user performance into Excellent,Good, Average, and Poor categories, making the analysis simple and easy to understand.

Interpretation

The analysis shows how the number of likes can be used as an indicator of user engagement. Users with higher numbers of likes are classified as having better performance, while users with fewer likes receive lower performance categories. This helps identify differences in social media engagement among users.

Achievement of Objectives

The project successfully achieved its objectives by implementing social media data processing, individual user analysis, likes-based performance classification, and report generation. The system provides a simple and efficient method for analysing and interpreting social media data.

Implications

The Social Media Data Analyser can be useful for students, content creators, social media managers, and researchers to understand user engagement and social media performance. In the future, the system can be extended with comments, shares, followers, sentiment analysis, engagement rates, graphics, machine learning-based

6. Mapping POs, PSOs and SDGs

6.1 Program Outcomes (POs) Mapping

PO No.	Program Outcome	Project Relevance
PO1	Engineering Knowledge	Applied Python programming, object-oriented programming, data structures, basic mathematics and GUI concepts to develop the Social Media Data Analyser.
PO2	Problem Analysis	Analysed the problem of manually tracking social media engagement and converted it into user inputs, likes/comments/shares, rules and reports.
PO3	Design/Development of Solutions	Designed user and post classes, GUI tabs, validation rules, engagement calculations and likes-based performance classification to solve the identified problem.
PO4	Conduct Investigations of Complex Problems	Tested the application using different users, posts, valid inputs and invalid inputs, and checked whether the generated analysis and reports were correct.
PO5	Engineering Tool Usage	Used Python 3, Tkinter/ttk, object-oriented programming concepts and VS Code for implementation, testing and debugging.
PO6	The Engineer and Society	Provides a simple way to understand social-media engagement and user performance, supporting responsible interpretation of digital activity.
PO7	Environment and Sustainability	Uses a software-based approach for analysis and reporting, reducing the need for manual paper-based tracking and supporting digital workflows.
PO8	Ethics	Encourages structured handling of user and engagement information and highlights the importance of using social-media data responsibly.
PO9	Individual and Team Work	The project was developed by two students who shared design, coding, testing and documentation responsibilities.
PO10	Communication	The GUI presents results through tables, analysis screens and individual reports, while the project report documents the implementation and findings clearly.
PO11	Life-Long Learning	The project strengthened learning of Tkinter/ttk widgets, OOP, dictionaries, exception handling, data processing and GUI development.
PO12	Project Management and Finance	Required planning of project requirements, coding tasks, testing and documentation within the scope and resources of a course-end project.

6.2 Program Specific Outcomes (PSOs) Mapping

PSO No.	Program Specific Outcome	Project Relevance
PSO1	Apply programming and computing fundamentals to develop software solutions.	The project applies Python, OOP, data structures, conditional logic, functions, exception handling and GUI programming to build a working analyser.
PSO2	Analyse, design and implement computing applications for real-world problems.	The system models social-media users and posts, calculates engagement, classifies performance and generates individual user reports.
PSO3	Use modern computing tools and technologies for software development and data analysis.	Python 3, Tkinter/ttk and VS Code are used to implement, test and demonstrate the Social Media Data Analyser.

The PSO mapping is kept separate from the PO mapping so that the project contribution to Computer Science and Engineering-specific skills is clearly visible.

6.3 Sustainable Development Goals (SDGs) Mapping

SDG No.	Goal	Project Relevance
SDG 4	Quality Education	The project is an educational software-development exercise that develops programming, problem-solving, analytical and technical skills.
SDG 8	Decent Work and Economic Growth	The project develops practical programming and data-analysis skills relevant to IT, software development, data science and digital technology careers.
SDG 9	Industry, Innovation and Infrastructure	The project demonstrates software-based data analysis and digital innovation by converting social-media engagement data into structured performance reports.
SDG 12	Responsible Consumption and Production	The application uses a digital workflow for data entry, analysis and reporting, supporting efficient information handling with minimal physical resources.

Overall, the project most directly supports SDG 4, SDG 8 and SDG 9 through technical learning, employability-oriented skills and software-based innovation.

Conclusion

This project successfully implemented a Social Media Data Analyser capable of processing social media data and generating meaningful performance reports for individual users. The system analyses user-related information such as the number of posts, likes, comments, and other available engagement data to evaluate overall social media performance. Based on the number of likes received, the system categorizes individual users into Excellent, Good, Average, and Poor performance levels. This classification makes it easier to understand user engagement and quickly identify users with high or low social media activity.

The project demonstrates the practical use of Python programming, data processing, conditional statements, file handling, and basic data analysis techniques to convert raw social media information into useful results. Instead of manually examining large amounts of social media data, the analyser provides a simple and systematic approach for generating user-specific reports. It can help in identifying engagement patterns, comparing user performance, and understanding the popularity of individual users or posts.

The system also provides a foundation for developing more advanced social media analytics applications. The current implementation focuses mainly on likes-based classification, making it simple, easy to understand, and suitable for basic performance analysis. The modular nature of the system allows additional parameters and analytical techniques to be incorporated in the future without completely redesigning the application.

Future Enhancements

For future development, the Social Media Data Analyser can be enhanced by including comments, shares, followers, hashtags, views, engagement rate, posting frequency, and post-performance comparison. Sentiment analysis can be added to determine whether user comments and posts are positive, negative, or neutral. Machine learning techniques can also be incorporated to predict future engagement, identify trending content, and detect patterns in user behaviour.

The system can further be improved by developing interactive dashboards, graphical charts, real-time data collection, platform integration, and automated report generation. Features such as user ranking, personalized recommendations, trending-topic detection, and advanced engagement analysis can make the application more powerful. Integration with social media APIs could allow the system to analyse real-time data instead of relying only on stored datasets.

Overall, the project provides a useful and practical approach to social media data analysis and demonstrates how programming and data analytics can be applied to real-world digital platforms. With further development, the system can be transformed into a more comprehensive analytical tool that supports better understanding, comparison, prediction, and decision-making based on social media data.

Bibliography

Online Resources
[1] **Python Documentation** – Python Programming Language and Data Processing
https://docs.python.org/3/
[2] **GeeksforGeeks** – Python Data Analysis and Text Processing
https://www.geeksforgeeks.org/python/
[3] **Pandas Documentation** – Data Analysis and Manipulation Using Python
https://pandas.pydata.org/docs/
[4] **Matplotlib Documentation** – Data Visualization Using Python
https://matplotlib.org/stable/
[5] **Scikit-learn Documentation** – Machine Learning and Text Analysis Techniques
https://scikit-learn.org/stable/
Research Article References
[1] Liu, B. (2012). *Sentiment Analysis and Opinion Mining*. Morgan & Claypool Publishers.
[2] Pang, B., & Lee, L. (2008). *Opinion Mining and Sentiment Analysis*. Foundations and Trends in Information Retrieval.
[3] Kouloumpis, E., Wilson, T., & Moore, J. D. (2011). *Twitter Sentiment Analysis: The Good the Bad and the OMG!*. Proceedings of the Fifth International AAAI Conference on Weblogs and Social Media.
[4] Go, A., Bhayani, R., & Huang, L. (2009). *Twitter Sentiment Classification using Distant Supervision*. Stanford University.
[5] Gundecha, P., & Liu, H. (2012). *Mining Social Media: A Brief Introduction*. Springer.